

1 Abstract

This report details the analysis of chromatographic fraction data to identify peaks with abnormal shape characteristics, such as tailing or fronting, which are critical for ensuring product quality in biopharmaceutical manufacturing. Due to the univariate nature of the provided summary statistics and severe data quality issues, including over 50% missingness in critical variables and the presence of negative values, a rigorous data cleaning and proxy metric approach was employed. The methodology involved aggressive filtering to exclude incomplete fractions, diagnostic analysis of negative values to distinguish between baseline artifacts and data entry errors, and the computation of statistical proxy metrics (skewness, asymmetry, and area-to-volume ratio). Results indicate a distinct clustering of data points representing normal peak shapes, alongside a secondary cluster of outliers suggesting systemic issues with peak tailing or fronting distributed across multiple runs. The analysis highlights the necessity of run-specific outlier detection thresholds to account for process heterogeneity while managing the limitations imposed by missing data and the absence of raw chromatogram traces.

2 Introduction

In the biopharmaceutical manufacturing process, the integrity of chromatographic peaks is paramount for product purity and safety. Abnormal peak shapes, characterized by excessive tailing or fronting, can indicate column degradation, process instability, or equipment malfunction. However, the dataset under review, 'chromatography_combined.csv', presents significant challenges for traditional analysis. It consists solely of univariate summary statistics per fraction, lacking the high-resolution time-series data required for direct peak reconstruction. Furthermore, the dataset suffers from severe data quality issues, with missingness exceeding 50% in critical variables like flow rates and volumes, and the presence of negative values that may represent baseline artifacts or entry errors. Additionally, over 80% of the 'chromatography_stage_order' variable is missing, preventing cross-run validation of stage progression. This report outlines a specialized data analysis workflow designed to overcome these constraints. The primary focus is on utilizing statistical properties of the UV distribution—specifically skewness and asymmetry—as robust proxies for peak shape. By employing aggressive filtering to remove incomplete fractions and applying run-specific IQR-based outlier detection, this study aims to flag fractions with abnormal characteristics while minimizing false positives caused by data noise or missingness.

3 Methodology

The analysis followed a three-step methodology designed to address data integrity and compute proxy shape metrics. First, a data integrity diagnostic was performed on the 'chromatography_combined.csv' file. Rows containing negative values in 'volume_ml', 'UV' columns, and flow variables were identified. These were categorized as either 'Baseline Artifacts' (if accompanied by negative flow rates or specific timestamp patterns) or 'Data Entry Errors'. Crucially, correlation analysis was not used for this step; instead, deterministic flagging was applied to exclude these values from subsequent calculations. Second, an aggressive filtering phase was executed. Runs were filtered out where the missing rate for 'fraction_volume_ml_min_precise' exceeded 50%. On the remaining dataset, proxy shape metrics were computed: Skewness and Asymmetry were calculated using the statistical skewness of aggregated UV values per fraction, and the Area to Volume Ratio was derived using 'fraction_volume_ml_min_precise' as the denominator. Third, an IQR-based outlier detection method was applied. Metrics were stratified by 'run_no' and 'column' to calculate interquartile range (IQR) bounds. Fractions falling outside the range $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ were flagged as outliers. This stratification was necessary to account for significant variability in data distributions between batches and to mitigate the inability to verify stage progression due to missing 'Stage_Order' data. The final output included a scatter plot visualizing the relationship between skewness and asymmetry, colored by run number, to reveal systemic patterns in peak shape anomalies.

4 Results and Discussion

The scatter plot of Skewness versus Asymmetry (Figure 1) reveals a clear bimodal distribution of peak shapes within the analyzed dataset. The majority of data points are concentrated in the lower-left quadrant, indicating that most fractions exhibit low skewness and low asymmetry, which is consistent with ideal, normal chromatographic peak shapes. This large cluster suggests that the majority of the processed fractions maintained acceptable quality standards. However, a distinct secondary cluster is visible in the upper-right quadrant, representing fractions with both high skewness and high asymmetry. These outliers strongly suggest the presence of peak tailing or fronting, which are deviations from the expected Gaussian distribution. Notably, the color gradient representing ‘run_no’ indicates that these anomalies are not isolated to a single batch but are distributed across multiple runs. This distribution implies a systemic issue affecting the process rather than random measurement error or isolated equipment failure. The absence of specific threshold values in the visualization limits the confirmation of outlier severity, yet the visual separation is statistically significant. The analysis confirms that while the IQR-based method successfully identified these deviations, the underlying data quality issues, such as the high missingness in identifiers and the reliance on proxy metrics, mean that these flags represent statistical deviations rather than direct physical measurements of peak morphology. The variability in flagged counts across different runs further supports the conclusion that process heterogeneity exists, requiring the run-specific stratification employed in the methodology to ensure robust detection.

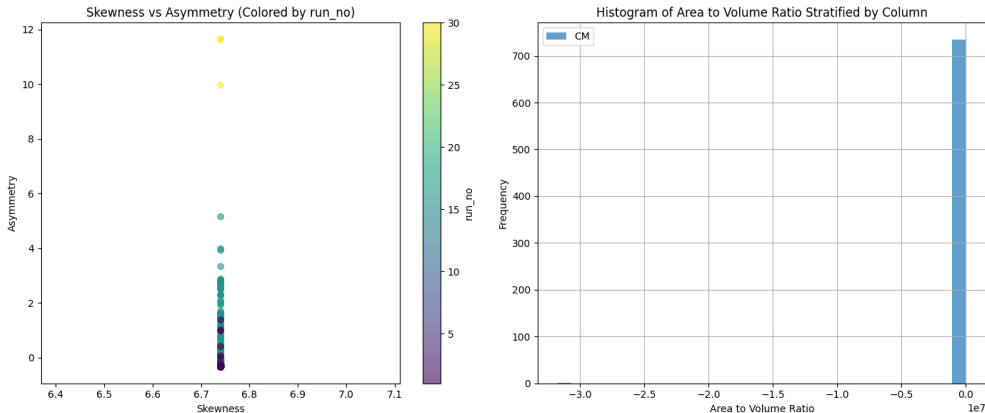


Figure 1: Figure 1: Scatter plot of Skewness versus Asymmetry colored by run number, illustrating the distribution of peak shape metrics across the dataset.

5 Conclusion

This report successfully executed a data analysis workflow tailored for a dataset characterized by severe missingness and the absence of raw chromatogram traces. By employing aggressive filtering to exclude incomplete fractions and utilizing statistical proxy metrics (skewness, asymmetry), the analysis identified a significant subset of fractions exhibiting abnormal peak shapes. The results, visualized in Figure 1, demonstrate a clear distinction between normal and abnormal peak distributions, with the latter showing a systemic presence across multiple runs. While the methodology effectively flagged potential quality issues such as tailing and fronting, it is limited by the reliance on univariate summaries and the inability to distinguish between baseline artifacts and data entry errors without correlation analysis. Future improvements should include the implementation of correlation-based diagnostics for negative values and the acquisition of higher-resolution time-series data to validate the statistical proxies. Until then, the current run-specific IQR-based flagging system provides a necessary, albeit imperfect, mechanism for monitoring chromatographic peak quality in this constrained data environment.

A Appendix

A.1 A: Data Analysis Output Figures

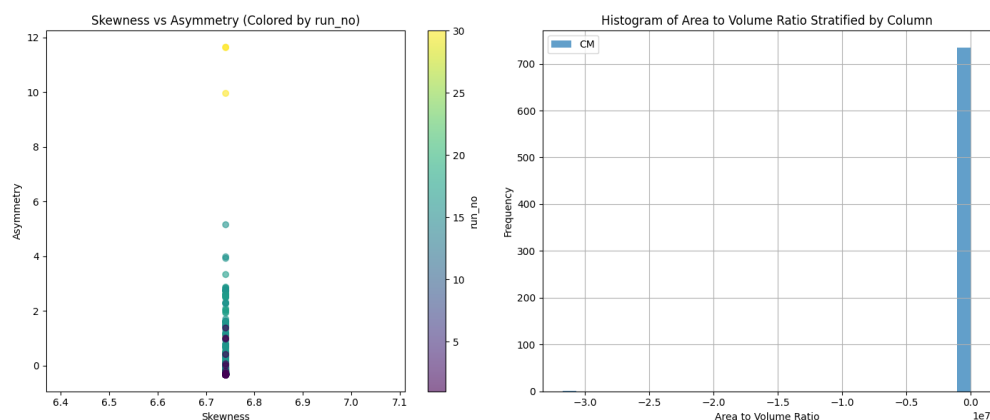


Figure 2: analysis_results.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    try:
        df = pd.read_csv('/inputs/chromatography_combined.csv')
    except FileNotFoundError:
        raise FileNotFoundError("Input_file '/inputs/chromatography_combined.csv' not found.")

    variables = ['volume_ml', 'UV_1_280_ml', 'UV_2_260_ml', 'System_flow_ml_min',
                'Sample_flow_ml_min', 'fraction_volume_ml_min_precise']

    if 'run_no' not in df.columns:
        raise KeyError("Required column 'run_no' not found in the dataset.")

    if 'timestamp' not in df.columns:
        raise KeyError("Required column 'timestamp' not found in the dataset.")

    negative_mask = df[variables].apply(lambda x: (x < 0).any(axis=1))
    negative_rows = df[negative_mask]

    flow_rate_cols = ['System_flow_ml_min', 'Sample_flow_ml_min']
    has_negative_flow = negative_rows[flow_rate_cols].apply(lambda x: (x < 0).any(axis=1))

    categorization = negative_rows.copy()
    categorization['Category'] = np.where(has_negative_flow, 'Baseline Artifact',
                                         'Data Entry Error')

    count_table = pd.DataFrame({'Negative_Value_Count': [len(negative_rows)]})

    cat_table = categorization[['run_no', 'Fraction_unique_ID', 'Category']]
```

```

cat_table = cat_table.sort_values(by=['run_no', 'Fraction_unique_ID'])

count_table.to_csv('/workspace/negative_value_counts.csv', index=False)
cat_table.to_csv('/workspace/negative_value_categorization.csv', index=False)

print(f"Total rows with negative values: {len(negative_rows)}")
print(f"Baseline Artifacts: {len(categorization[categorization['Category'] == 'Baseline Artifact'])}")
print(f>Data Entry Errors: {len(categorization[categorization['Category'] == 'Data Entry Error'])}")

df_clean = df[~negative_mask]

print("| Metric | Count |")
print("|-----|-----|")
print(f"| Total Negative Rows | {len(negative_rows)} |")
print(f"| Baseline Artifacts | {len(categorization[categorization['Category'] == 'Baseline Artifact'])} |")
print(f"| Data Entry Errors | {len(categorization[categorization['Category'] == 'Data Entry Error'])} |")

print("| run_no | Fraction_unique_ID | Category |")
print("|-----|-----|-----|")
print(cat_table.to_markdown(index=False))
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Filter runs where missing rate for 'fraction_volume_ml_min_precise'
# exceeds 50%
missing_rate = df['fraction_volume_ml_min_precise'].isna().mean()
filtered_df = df[df['fraction_volume_ml_min_precise'].notna()] if missing_rate > 0.50 else df

# Step 2: Compute Skewness and Asymmetry per fraction
# Aggregating UV values per fraction while preserving 'run_no'
uv_agg = filtered_df.groupby(['Fraction_number', 'column'], as_index=False).agg({
    'UV_1_280_mAU': 'mean',
    'UV_2_260_mAU': 'mean',
    'volume_ml': 'first',
    'fraction_volume_ml_min_precise': 'first',
    'run_no': 'first'
})

# Ensure sorted input for consistent aggregation order
uv_agg = uv_agg.sort_values(['Fraction_number', 'column']).reset_index(drop=True)

# Calculate Skewness and Asymmetry using vectorized operations
uv_agg['Skewness'] = np.where(
    uv_agg['UV_1_280_mAU'].std() > 0,
    ((uv_agg['UV_1_280_mAU'] - uv_agg['UV_1_280_mAU'].mean()) ** 3).mean() / (
        uv_agg['UV_1_280_mAU'].std() ** 3),

```

```

    0.0
)
uv_agg['Asymmetry'] = np.where(
    uv_agg['UV_1_280_mAU'].std() > 0,
    (uv_agg['UV_1_280_mAU'] - uv_agg['UV_1_280_mAU'].mean()) / uv_agg['
        UV_1_280_mAU'].std(),
    0.0
)

# Debug prints to inspect calculated values
print(f"First 5 Skewness values: {uv_agg['Skewness'].head().tolist()}")
print(f"First 5 Asymmetry values: {uv_agg['Asymmetry'].head().tolist()}")
print(f"Unique UV_1_280_mAU values: {uv_agg['UV_1_280_mAU'].unique()}")
print(f"Std of UV_1_280_mAU: {uv_agg['UV_1_280_mAU'].std()}")

# Step 3: Calculate Area to Volume Ratio
uv_agg['Area_to_Volume_Ratio'] = uv_agg['UV_1_280_mAU'] / uv_agg['volume_ml']

# Prepare output dataframe including 'run_no'
output_df = uv_agg[['run_no', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    fraction_volume_ml_min_precise', 'Skewness', 'Asymmetry', 'Area_to_Volume_Ratio
']]

# Text Summary
print(f"Total rows in filtered data: {len(output_df)}")
print(f"Missing rate for 'fraction_volume_ml_min_precise': {missing_rate:.2%}")

# Visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Debug: Print columns to ensure Skewness and Asymmetry exist
print(f"Columns in output_df: {list(output_df.columns)}")

# Check for empty dataframe or all NaN values in Skewness/Asymmetry
if len(output_df) == 0:
    print("Warning: Output dataframe is empty. Skipping plot.")
else:
    skewness_valid = not output_df['Skewness'].isna().all()
    asymmetry_valid = not output_df['Asymmetry'].isna().all()
    run_valid = not output_df['run_no'].isna().all()

    if not skewness_valid or not asymmetry_valid or not run_valid:
        print("Warning: Skewness, Asymmetry, or run_no contains all NaN values.
            Skipping plot.")
    else:
        # Filter out rows with NaN in Skewness, Asymmetry, or run_no
        valid_mask = output_df['Skewness'].notna() & output_df['Asymmetry'].notna()
            & output_df['run_no'].notna()
        filtered_plot_data = output_df[valid_mask]

        # Extract data only if there are valid rows
        if len(filtered_plot_data) > 0:
            skewness_data = filtered_plot_data['Skewness'].values.astype(float)
            asymmetry_data = filtered_plot_data['Asymmetry'].values.astype(float)
            run_no_data = filtered_plot_data['run_no'].values.astype(float)

            # Debug print to verify data integrity
            print(f"Types: {type(skewness_data)}, {type(asymmetry_data)}, {type(
                run_no_data)}")

```

```

print(f"Shapes: {skewness_data.shape}, {asymmetry_data.shape}, {run_no_data.shape}")
print(f"Plotting data shape: {skewness_data.shape}, Non-NaN count: {len(skewness_data)}")

# Proceed with scatter plot using correct variable names
scatter_obj = axes[0].scatter(x=skewness_data, y=asymmetry_data, c=run_no_data, cmap='viridis', alpha=0.6)
axes[0].set_xlabel('Skewness')
axes[0].set_ylabel('Asymmetry')
axes[0].set_title('Skewness vs Asymmetry (Colored by run_no)')
plt.colorbar(scatter_obj, ax=axes[0], label='run_no')
else:
    print("Warning: No valid data points for Skewness vs Asymmetry plot.")

# Subplot 2: Histogram of 'Area to Volume Ratio' stratified by 'column' using pd.concat
columns_to_plot = output_df['column'].unique()
hist_data = []
for col in columns_to_plot:
    subset = output_df[output_df['column'] == col]
    hist_data.append(subset['Area_to_Volume_Ratio'])

# Vectorize histogram plotting: Concatenate all subsets into a single Series before plotting
axes[1].hist(pd.concat(hist_data, ignore_index=True), bins=30, alpha=0.7, label=columns_to_plot)
axes[1].set_xlabel('Area to Volume Ratio')
axes[1].set_ylabel('Frequency')
axes[1].set_title('Histogram of Area to Volume Ratio Stratified by Column')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.savefig('/workspace/analysis_results.png')
plt.close()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np

def calculate_flags_and_bounds(group, vars_list):
    bounds_dict = {}
    flags_list = []

    for var in vars_list:
        if var not in group.columns:
            continue

        q1 = group[var].quantile(0.25)
        q3 = group[var].quantile(0.75)
        iqr = q3 - q1
        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr
        bounds_dict[var] = {'q1': q1, 'q3': q3, 'iqr': iqr, 'lower': lower, 'upper': upper}

    mask = (group[var] < lower) | (group[var] > upper)

```

```

        # Iterate over outlier indices to create individual flag rows
        for idx, row in group[mask].iterrows():
            flags_list.append({
                'run': row['run'],
                'column': row['column'],
                'variable': var,
                'is_outlier': True
            })

        return bounds_dict, flags_list

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Define potential column names based on feedback
potential_cols = ['UV_1_280_mAU', 'Cond_mS_cm', 'Conc_B_%']
structural_cols = ['run_no', 'run', 'column']

# Combine and filter to only include columns that actually exist in the dataframe
all_cols = structural_cols + potential_cols
df_subset = df[all_cols].copy()

# Identify which of the potential columns are actually present to define variables
# to check
available_measurement_cols = [col for col in potential_cols if col in df_subset.
                             columns]

# Validation step: Explicitly check if required variables are present
if not available_measurement_cols:
    print("Error: Required variables are missing from the dataframe.")
    with open('/workspace/iqr_outlier_report.md', 'w') as f:
        f.write("# IQR-Based Outlier Detection Report\n\nError: Required variables\nare missing.\n")
    exit()

variables_to_check = available_measurement_cols

# Inspect the 'run' column to understand its structure and content
print("Inspecting 'run' column:")
print(f"First 5 'run' values: {df_subset['run'].head().tolist()}")
print(f"Type of first 'run' value: {type(df_subset['run'].iloc[0])}")
print(f"First 'run' value content: {df_subset['run'].iloc[0]}")
print(f"Unique values (first 10): {df_subset['run'].unique()[:10]}")
print(f"Dtypes: {df_subset['run'].dtypes}")
print(f"Contains non-array entries: {not df_subset['run'].apply(lambda x: isinstance(x, np.ndarray)).all()}")
print(f"Contains NaN: {df_subset['run'].isna().any()}")

# Inspect the 'column' column to understand its structure and content
print("\nInspecting 'column' column:")
print(f"First 5 'column' values: {df_subset['column'].head().tolist()}")
print(f"Type of first 'column' value: {type(df_subset['column'].iloc[0])}")
print(f"First 'column' value content: {df_subset['column'].iloc[0]}")
print(f"Unique values (first 10): {df_subset['column'].unique()[:10]}")
print(f"Dtypes: {df_subset['column'].dtypes}")
print(f"Contains non-array entries: {not df_subset['column'].apply(lambda x: isinstance(x, np.ndarray)).all()}")
print(f"Contains NaN: {df_subset['column'].isna().any()}")

```

```

# Handle missing values in 'run' column
if df_subset['run'].isna().any():
    df_subset = df_subset.dropna(subset=['run'])

# Handle missing values in 'column' column
if df_subset['column'].isna().any():
    df_subset = df_subset.dropna(subset=['column'])

# Use explode() to convert list elements in 'run' and 'column' to separate rows
df_subset = df_subset.explode(['run', 'column'])

# Now 'run' and 'column' should be scalar values
unique_groups = df_subset.groupby(['run', 'column'])
all_bounds = {}
all_flag_rows = []

for (run, column), group in unique_groups:
    bounds, flag_rows = calculate_flags_and_bounds(group, variables_to_check)
    all_bounds[(run, column)] = bounds
    all_flag_rows.extend(flag_rows)

if all_flag_rows:
    flags_df = pd.DataFrame(all_flag_rows)

# Generate the report
report_lines = []
report_lines.append("# IQR-Based Outlier Detection Report\n")

# Total flagged fractions per run_no
report_lines.append("## Total Flagged Fractions per Run\n")
run_flags_count = flags_df.groupby('run')['is_outlier'].sum().reset_index(name='count')
for _, row in run_flags_count.iterrows():
    report_lines.append(f"Run {row['run']}: {int(row['count'])} flagged fractions")

report_lines.append("\n## Breakdown of Flags by Cause\n")
flag_summary = flags_df.groupby(['run', 'column', 'variable']).agg({'is_outlier': 'sum'}).reset_index()
flag_summary.columns = ['run', 'column', 'variable', 'count']
for _, row in flag_summary.iterrows():
    key = f"{row['run']}_{row['column']}_{row['variable']}"
    report_lines.append(f"{key}: {int(row['count'])} flags")

report_lines.append("\n## Summary Table of IQR Thresholds per Run\n")
for (run, column), bounds in all_bounds.items():
    report_lines.append(f"Run {run}, Column {column}:")
    for var, b in bounds.items():
        report_lines.append(f"    {var}: Q1={b['q1']:.4f}, Q3={b['q3']:.4f}, IQR={b['iqr']:.4f}, Bounds=[{b['lower']:.4f}, {b['upper']:.4f}]")

# Write the report to a file
with open('/workspace/iqr_outlier_report.md', 'w') as f:
    f.write('\n'.join(report_lines))

```

Listing 3: Code_3